

El Paradigma Lógico

Introducción

El **paradigma lógico** es un paradigma de programación declarativa, basado en el cálculo de predicados (lógica de primer orden) aunque no captura todo su poder expresivo.

Se describen las características del resultado deseado pero no se especifica una manera concreta para alcanzarlo.

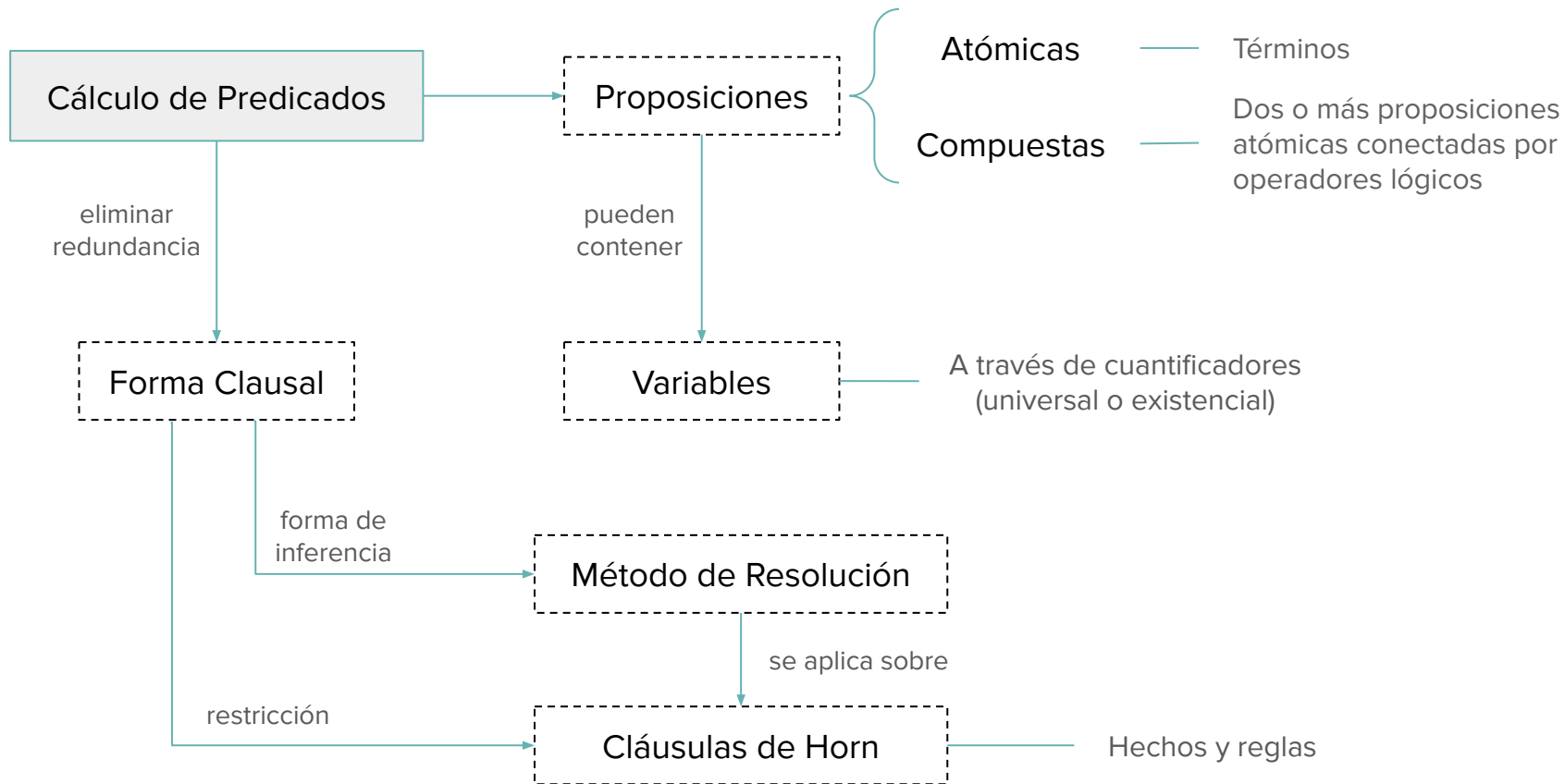
Las variables representan valores, no locaciones de memoria.

El programa consiste en una descripción del resultado deseado.

Mecanismo de inferencia basado en algoritmo de resolución y unificación.

Prolog es el lenguaje representativo del paradigma.

Cálculo de Predicados



Cálculo de Predicados

Una **proposición** es una sentencia lógica que puede ser verdadera o falsa, y consiste de objetos que pueden estar relacionados.

- **Proposiciones atómicas**, formadas por términos. Un **término** puede ser una constante, una variable o un término compuesto $f(t1, t2, ..., tn)$.
- **Proposiciones compuestas**, formadas por dos o más proposiciones atómicas conectadas por operadores lógicos.

El **cálculo de predicados** nos permite modelar problemas y razonar sobre ellos a través de proposiciones.

- Expresar proposiciones.
- Expresar relaciones entre proposiciones por medio de reglas.
- Deducir nueva información a partir de proposiciones existentes y sus relaciones.

Proposiciones Atómicas

a

$p(b)$

$q(c, d)$

$\text{padre}(\text{homero}, \text{lisa})$

$\text{humano}(\text{homero})$

$\text{mortal}(\text{homero})$

Proposiciones Compuestas

$\neg a$

$p(b) \cap q(c, d)$

$\neg a \cup p(b) \cup q(c, d)$

$a \supset b \equiv \neg a \cup b$

$(\exists X)(\text{humano}(X) \supset \text{mortal}(X))$

$(\forall X)(\text{humano}(X) \supset \text{mortal}(X))$

Semántica por Valores

El paradigma lógico utiliza el concepto matemático de variable, por lo cual **una variable es una abstracción de un valor**. En este caso, **los valores ligados a un identificador son términos**.

El valor que puede tomar una variable puede ser cualquier término. Una vez ligado un valor, el mismo no puede cambiar.

Mientras no esté ligada a un valor, se dice que la variable está *libre*.

```
p(a).
```

```
?- p(X).
```

```
X = a
```

```
p(X) ← q(a), r(X).
```

```
q(a).
```

```
r(lista(1, 2, 3)).
```

```
?- p(X).
```

```
X = lista(1, 2, 3)
```

No existe una locación de memoria asociada a una variable.

Cálculo de Predicados :: Forma Clausal

Un **problema con el cálculo de predicados** es que permite expresar distintas proposiciones que tienen el mismo significado.

Una forma estándar de expresar proposiciones es la **forma clausal**. Una proposición en forma clausal tiene la siguiente forma general:

$$\underbrace{B_1 \cup B_2 \cup \dots \cup B_n}_{\text{Consecuente}} \subset \underbrace{A_1 \cap A_2 \cap \dots \cap A_m}_{\text{Antecedente}} \quad (\text{donde } A_i \text{ y } B_i \text{ son términos})$$

La forma clausal nos permite tener una forma estándar sin redundancia que mantiene el poder expresivo del cálculo de predicados.

Cálculo de Predicados :: Forma Clausal

Algunas características interesantes de la **forma clausal**:

- No se requieren cuantificadores existenciales.
- Los cuantificadores universales quedan implícitos.
- Únicamente son necesarios los operadores de conjunción, disyunción y negación.
- Todas las proposiciones del cálculo de predicados pueden ser expresadas en forma clausal.
- Existe un algoritmo que permite convertir cualquier proposición del cálculo de predicados a forma clausal.

Mecanismo de Inferencia

El mecanismo de inferencia permite **derivar nuevo conocimiento** (nuevas proposiciones a partir de un conjunto de proposiciones iniciales expresadas en forma clausal) **y hallar soluciones**.

El proceso de **resolución** consiste de 3 pasos:

1. Los términos en el lado izquierdo de las proposiciones iniciales se conectan con un operador de disyunción. Forman el lado izquierdo de la nueva proposición.
2. Los términos en el lado derecho de las proposiciones iniciales se conectan con un operador de conjunción. Forman el lado derecho de la nueva proposición.
3. Cualquier término que aparezca en ambos lados en la nueva proposición es removido de ambos lados.

Proposiciones iniciales

$C \cup E \leftarrow A \cap B$ (*A y B implica C o E*)

$D \leftarrow C \cap B$ (*C y B implica D*)

Lado izquierdo de la nueva proposición

$C \cup E \cup D$

Lado derecho de la nueva proposición

$A \cap B \cap C$

Nueva proposición

$D \cup E \leftarrow A \cap B$ (*A y B implica D o E*)

Mecanismo de Inferencia :: Unificación

El proceso de determinar los valores adecuados para las variables libres en una proposición se denomina **unificación**.

Para que dos términos unifiquen:

- Deben tener el mismo functor y la misma aridad.
- Los argumentos deben unificar uno a uno, manteniendo las ligaduras que se produzcan.
 - Una variable siempre unifica con un término, quedando ligada a dicho término.
 - Dos variables siempre unifican entre sí. Cuando una se liga a un término, la otra también se liga.

$p(X)$
 $q(X)$



$p(X, f(b))$

$p(X, f(b), c)$



$p(X, f(b), c)$

$p(X, g(b), c)$



$p(X, f(b))$

$p(Y, f(Z))$

$p(a, f(b))$



$[X = a, Z = b, Y = a]$

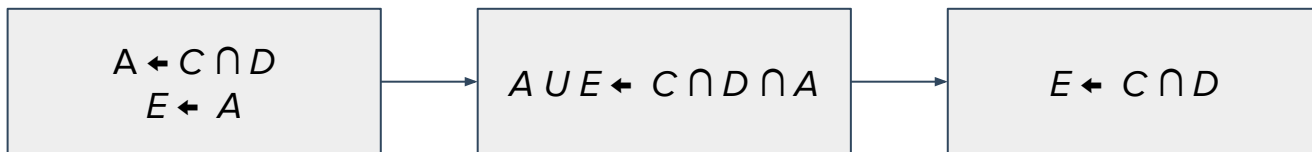
Cálculo de Predicados :: Cláusulas de Horn

Por motivos de computabilidad, se impone una restricción adicional: las proposiciones deben ser **cláusulas de Horn**.

$$\underbrace{H}_{\text{Cabeza}} \leftarrow \underbrace{G1 \wedge G2 \wedge \dots \wedge Gm}_{\text{Cuerpo}}$$

Las cláusulas de Horn contienen una única proposición atómica en el lado izquierdo.

Pese al menor poder expresivo, vamos a utilizar cláusulas de Horn para expresar proposiciones (hechos o reglas) e inferir nuevas proposiciones aplicando el proceso de resolución.



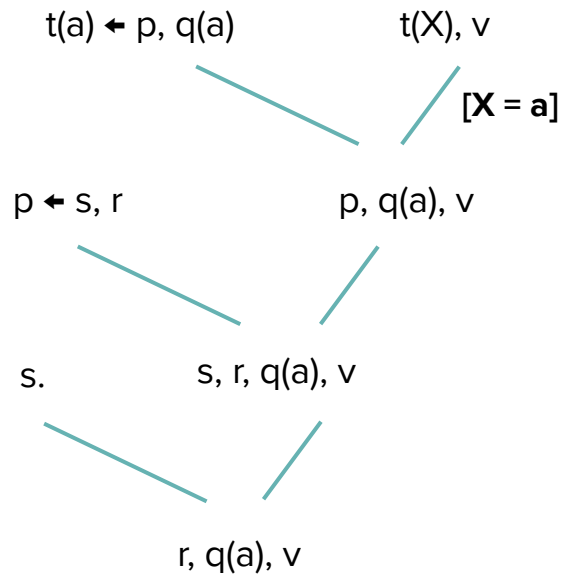
Ejemplo

Programa

$t(a) \leftarrow p, q(a)$
 $p \leftarrow s, r$
 $s.$
 $r.$
 $q(a).$
 $v.$

Consulta

$t(X), v ?$



Reversibilidad

Los argumentos de los predicados pueden actuar indistintamente como argumentos de entrada o de salida.

Los argumentos de entrada deben ser instanciados, mientras que los de salida son variables libres.

```
padre(juan, jorge)
padre(javier, marcos)
```

```
padre(juan, X)      ⇒ [X = jorge]
padre(X, jorge)     ⇒ [X = juan]
padre(X, Y)         ⇒ [X = juan, Y = jorge]
                    ⇒ [X = javier, Y = marcos]
```

Inexistencia de Tipos

No se provee la noción de tipo de dato que puede aparecer en otros paradigmas.

El programador debe encargarse de especificar restricciones en el programa para evitar un mal uso de términos y predicados. No vamos a contar con un sistema de tipos que haga los chequeos necesarios y nos aporte seguridad.

```
apilar(E, Pila, pila(E, Pila))
```

```
apilar(1, nil, X)  $\Rightarrow$  [X = pila(1, nil)]
```

```
apilar(2, pila(1, nil), Y)  $\Rightarrow$  [Y = pila(2, pila(1, nil))]
```

```
apilar(3, otra_cosa, Z)  $\Rightarrow$  [Z = pila(3, otra_cosa)]
```

```
apilar(E, Pila, pila(E, Pila))  $\leftarrow$  es_pila(Pila)
```

```
apilar(1, nil, X)  $\Rightarrow$  [X = pila(1, nil)]
```

```
apilar(2, pila(1, nil), Y)  $\Rightarrow$  [Y = pila(2, pila(1, nil))]
```

```
apilar(3, otra_cosa, Z)  $\Rightarrow$  error
```

El uso adecuado de los términos queda a cargo del programador.

En resumen...

Algunas propiedades que resultan importantes en el paradigma son:

- Se utilizan cláusulas de Horn para describir las características del problema (y de sus soluciones).
- Se aplica el algoritmo de resolución sobre cláusulas de Horn con el objetivo de derivar nueva información a partir del programa y encontrar soluciones para una consulta.
- Utiliza unificación para ligar las variables a valores cuando sea necesario en el proceso de resolución. Las variables se asocian a valores, no a locaciones de memoria (Semántica por valores).
- El mecanismo de inferencia permite predicados reversibles.
- No existe noción de tipos.
- El paradigma no impone ningún orden en el flujo de ejecución. La implementación del mecanismo de inferencia en un lenguaje particular va a imponer las restricciones necesarias.

Caso de Estudio: Prolog

Introducción

Prolog (Programmation en Logique) es el principal lenguaje representante del paradigma lógico.

Utiliza semántica por valores, no cuenta con tipos y el control de flujo se basa en resolución, unificación y backtracking.

Un programa Prolog se compone de tres clases de elementos: hechos, reglas y consultas.

Programa	Consulta
<pre>padre_de(homero, bart). padre_de(homero, lisa). padre_de(homero, maggie). madre_de(marge, bart). madre_de(marge, lisa). madre_de(marge, maggie). hermanos(X, Y) :- padre_de(Z, X), padre_de(Z, Y). hermanos(X, Y) :- madre_de(Z, X), madre_de(Z, Y).</pre>	<pre>?- padre_de(homero, bart). true ?- padre_de(bart, lisa). false ?- hermanos(bart, lisa). true</pre>

Ausencia de Tipos

En Prolog no existe la noción de tipos. Todas las restricciones deben ser impuestas como parte de la definición de los predicados.

La seguridad y confiabilidad del programa es responsabilidad del programador.

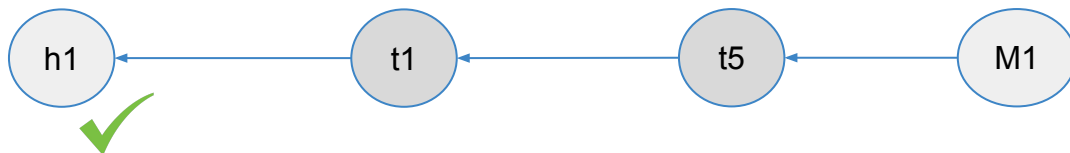
Programa	Consulta
<pre>suma(X, 0, X). suma(X, s(Y), s(Z)):- suma(X, Y, Z).</pre>	<pre>?- suma(s(algo), s(s(0)), Z). Z = s(s(s(algo)))</pre>
<pre>num(0). num(s(N)) :- num(N). suma(X, 0, X) :- num(X). suma(X, s(Y), s(Z)):- suma(X, Y, Z).</pre>	<pre>?- suma(s(algo), s(s(0)), Z). false ?- suma(s(0), s(s(0)), Z). Z = s(s(s(0)))</pre>

Estrategia de Búsqueda :: Control de Flujo

El no determinismo del cálculo de predicados no nos sirve si buscamos una forma efectiva de resolver consultas.

Prolog combina **backward-chaining con búsqueda en profundidad** para poder computar la solución a una consulta.

Backward chaining



Dado un conjunto de submetas, usando un algoritmo de **recorrido en profundidad** (*Depth First Search*), se encuentra una derivación completa para la primera submeta, antes de pasar a las siguientes submetas.

M1 :- t5.

t1 :- h1.

t2 :- h2, h3.

t3 :- t2.

t4 :- t6, t7.

t5 :- t1.

t6 :- h1, t2.

h1.

h2.

h3.

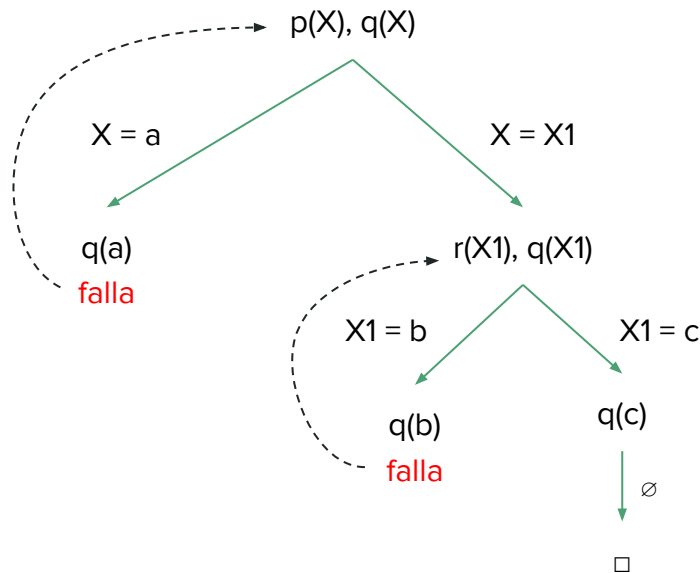
h4.

Estrategia de Búsqueda :: Backtracking

Finalmente, el **backtracking** es el mecanismo que utiliza Prolog para intentar satisfacer una submeta cuando falló un intento anterior.

Cuando una submeta falla, se vuelve a un punto de decisión anterior en la ejecución, donde las variables se instancian de una manera diferente. A partir de la nueva instanciación, se intenta verificar la submeta nuevamente.

Programa	Consulta
<code>p(a).</code> <code>p(X) :- r(X).</code> <code>r(b).</code> <code>r(c).</code> <code>q(c).</code>	<code>?- p(X), q(X).</code> <code>X = c</code>



Control Condicional usando Predicados

Prolog no cuenta con las sentencias condicionales habituales en los lenguajes imperativos.

El comportamiento condicional es definido usualmente mediante predicados por casos, donde cada caso respeta un cierto patrón distintivo.

Programa	Consulta
<pre>length([], 0). length([X Xs], Len) :- length(Xs, L), Len is L+1.</pre>	<pre>?- length([1, 2], L). L = 2</pre>
<pre>mayores([], _, []). mayores([X Xs], N, [X R]) :- X >= N, mayores(Xs, N, R). mayores([X Xs], N, R) :- X < N, mayores(Xs, N, R).</pre>	<pre>?- mayores([4, 7, 3], 5, L). L = [7]</pre>
<pre>distinguir_numero(X):- X >= 0, es_par(X), write(X), write("es natural par"). distinguir_numero(X):- X >= 0, write(X), write("es natural impar"). distinguir_numero(X):- write(X), write("es no es natural").</pre>	<pre>?- distinguir_numero(5). "5 es natural impar" true</pre>
	<pre>?- distinguir_numero(6). "6 es natural par" true</pre>

Recursión

Tampoco se cuenta con los mecanismos de iteración habituales de los lenguajes convencionales. **La manera de iterar es mediante la definición de predicados recursivos.**

Programa	Consulta
<pre>length([], 0). length([X Xs], Len) :- length(Xs, L), Len is L+1. member(X, [X _Xs]). member(X, [_Y Xs]) :- X\=Y, member(X, Xs). append([], Lista2, Lista2). append([X Xs], Lista2, [X R]) :- append(Xs, Lista2, R). conectados(X, Y) :- arco(X, Y). conectados(X, Y) :- arco(X, Z), conectados(Z, Y).</pre>	<pre>?- length([1, 2], L). L = 2 ?- member(1, [1, 2]). true ?- member(X, [1, 2]). X = 1; X = 2 ?- append([1], [2, 3], L). L = [1, 2, 3] ?- append([1], Lista2, [1, 5]). Lista2 = [5]</pre>

Reversibilidad

La **reversibilidad** permite definir predicados cuyos argumentos pueden actuar indistintamente como argumentos de entrada y/o salida.

Prolog utiliza el mecanismo de unificación para ligar variables libres a valores, lo que le permite tener reversibilidad.

Es un mecanismo más poderoso que el de pattern matching, en el cual la consulta debe estar totalmente instanciada.

Programa	Consulta
<pre>padre_de(homero, bart). padre_de(homero, lisa). padre_de(homero, maggie). madre_de(marge, bart). madre_de(marge, lisa). madre_de(marge, maggie). progenitor(X, Y):- padre_de(X, Y). progenitor(X, Y):- madre_de(X, Y).</pre>	<pre>?- progenitor(homero, X). X = bart; X = lisa; X = maggie. ?- progenitor(X, bart). X = homero; X = marge. ?- progenitor(X, Y). X = homero, Y = bart; X = homero, Y = lisa; ...</pre>

¿Qué lo aleja del Paradigma Lógico?

Control del Proceso de Resolución

En un ambiente de programación lógica puro, el orden es que se seleccionan las metas y en que se analiza la base de conocimiento es no determinista.

En Prolog se cuenta con una **estrategia de búsqueda predefinida**:

- La base de conocimiento se analiza de arriba hacia abajo.
- Las metas se seleccionan de izquierda a derecha.
- Se adopta backward-chaining con DFS y backtracking.

El programador debe tener en cuenta estas condiciones para asegurar la correctitud y mejorar la eficiencia del programa.

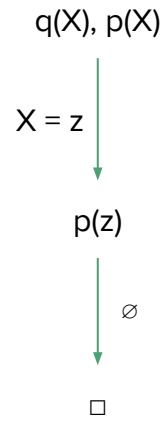
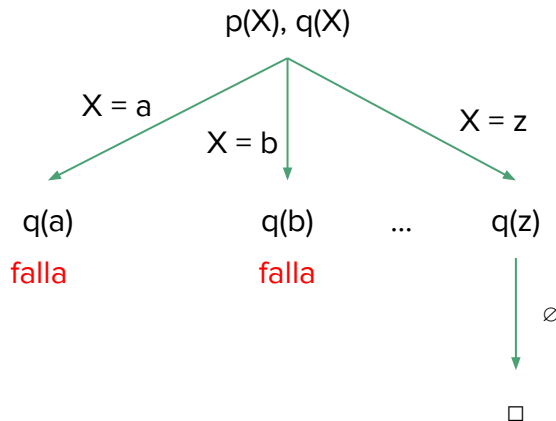
Programa	Consulta
<pre>arco(a, b). conectados(X, Y) :- conectados(X, Z), conectados(Z, Y). conectados(X, Y) :- arco(X, Y).</pre>	<pre>?- conectados(a, b). ...</pre>
<pre>arco(a, b). conectados(X, Y) :- arco(X, Y). conectados(X, Y) :- conectados(X, Z), conectados(Z, Y).</pre>	<pre>?- conectados(a, b). true</pre>

Control del Proceso de Resolución

El orden en que se especifican las metas en una consulta puede influir notablemente en la eficiencia de un programa.

El programador debe tener en cuenta el mecanismo que utiliza el lenguaje para resolver una consulta, y ordenar las metas de manera conveniente.

Programa	Consulta
$p(a).$ $p(b).$... $p(z).$ $q(z).$	$?- p(X), q(X).$ $X = z$ $?- q(X), p(X).$ $X = z$



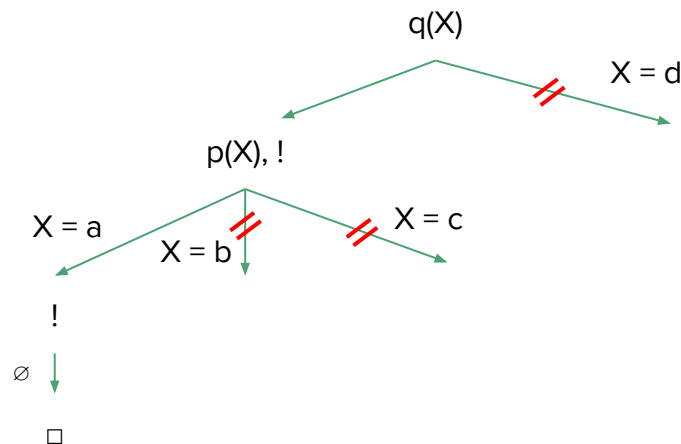
Control del Backtracking

El predicado predefinido **cut** (!) permite al programador **controlar el backtracking de manera explícita**.

El predicado siempre tiene éxito, y tiene el efecto colateral de podar el espacio de búsqueda, descartando las ramas alternativas del árbol de búsqueda que quedaron pendientes de ser exploradas al momento en que se comenzó a resolver la meta donde aparece el cut.

- Poda alternativas correspondientes a reglas que están debajo de la regla que contiene el cut.
- Poda soluciones alternativas de la meta conjuntiva a izquierda del cut.

Programa	Consulta
<code>p(a).</code> <code>p(b).</code> <code>p(c).</code> <code>q(X):- p(X), !.</code> <code>q(d).</code>	<code>?- q(X).</code> <code>X = a</code>



Suposición de Mundo Cerrado y Negación por Falla

Para Prolog, las únicas verdades son aquellas que puede probar a partir de la base de conocimiento del programa. Si recibe una meta que no puede probar como verdadera, entonces asume que es falsa. Esto se conoce como **suposición de mundo cerrado**.

Prolog utiliza **negación por falla** (*not*), una forma de negación diferente a la del cálculo de predicados.

```
not(P) :- call(P), !, fail.  
not(P).
```

Programa	Consulta
p(a). p(b).	?- p(c). false ?- not(p(c)). true

- El predicado *not(P)* tendrá éxito cuando no se encuentre una derivación para P.
- El predicado *not(P)* fallará cuando P pueda probarse verdadero.

Operadores Aritméticos y Relacionales

Las expresiones aritméticas son evaluadas usando el predicado **is**.

Es un operador en forma infija, con la siguiente forma:

Variable is Expresión

- *Expresión* es una expresión aritmética que debe estar complementamente instanciada.
- Si *Variable* está instanciada, el predicado retorna true si el valor coincide con el valor de la expresión, o false en caso contrario.
- Si *Variable* es libre, se liga al valor resultado de evaluar Expresión.

Los predicados relacionales ($>$, $<$, $>=$, $=<$, $=:=$, $=\backslash=$) evalúan sus argumentos de la misma forma que el predicado **is**.

El lenguaje pierde uniformidad ya que estos predicados no son reversibles.

Consulta

?- A is 4 + 5.

A = 9

?- 4 is 2 + 2.

true

?- 2 + 2 is 3 + 1.

false

?- 10 >= 3 + 4.

true

?- 2 + 2 =:= 3 + 1.

true

?- A is 4 + a.

error

Predicados *assert* y *retract*

Los predicados *assert* y *retract* permiten **modificar el programa de manera dinámica**. Con ello, podemos cambiar el comportamiento del programa.

El predicado *assert/1* permite agregar una nueva cláusula a la base de conocimiento (hecho o regla).

El predicado *retract/1* permite eliminar del programa una cláusula que unifique con su argumento.

Programa	Consulta
<pre><code>:- dynamic time/1 time(0). current_time(N):- time(N), M is N + 1, retract(time(_)), assert(time(M)).</code></pre>	<pre><code>?- current_time(T1), current_time(T2). T1 = 0 T2 = 1</code></pre>

Predicados findall y forall

El predicado ***findall*** tiene la siguiente signatura:

findall(Variable, Predicado, Lista)

Su efecto es el siguiente: Se forma una lista *Lista* conteniendo los valores de *Variable* con los cuales se verificado *Predicado*. El segundo argumento puede ser una meta conjuntiva.

Programa	Consulta
<pre>campo(astronomia). campo(psicologia). campo(matematicas). campo(literatura). interesa(carlos, astronomia). interesa(marcela, psicologia). interesa(diego, literatura). interesa(andrea, literatura).</pre>	<pre>?- findall(Nombre, (campo(literatura), interesa(Nombre, literatura)), ListaPersonasLiteratura). ListaPersonasLiteratura = [diego, andrea]</pre>

Predicados findall y forall

El predicado **forall** tiene la siguiente signatura:

forall(Condición, Acción)

Su efecto es el siguiente: Para cada posible unificación de *Condición*, se verifica *Acción*.

Resulta útil para verificar un predicado para una colección de elementos, o para modificar el programa de manera dinámica.

Programa	Consulta
<pre>:- dynamic hacer_curso/2 interesa(carlos, astronomia). interesa(marcelo, psicologia). interesa(diego, literatura). interesa(andrea, literatura).</pre>	<pre>?- forall(interesa(Nombre, Campo), assert(hacer_curso(Nombre, Campo))). true ?- hacer_curso(Persona, literatura). Persona = diego; Persona = andrea</pre>

Reflexividad

El predicado *clause(Head,Body)* busca una regla con cabeza *Head* y cuerpo *Body*. Con este predicado los programadores pueden tener acceso a todo el programa.

Prolog también nos provee el predicado *univ (Term =.. List)*, que permite transformar predicados en listas y viceversa.

Programa	Consulta
a. p :- q. aprueba(X):- estudia(X), rinde(X).	?- clause(a, Body). Body = true ?- clause(p, Body). Body = q ?- clause(aprueba(X), Body). Body = estudia(X), rinde(X) ?- (aprueba(X) :- estudia(X), rinde(X)) =.. List. List = [:-, aprueba(X), (estudia(X), rinde(X))]

Un lenguaje es **reflexivo** si permite a sus programas razonar acerca de su propia estructura.

Reflexividad y Meta-Programación

Un lenguaje es **reflexivo** si permite a sus programas razonar acerca de su propia estructura.

Podemos aprovechar la reflexibilidad para crear meta-programas, esto es, programas que pueden crear o manipular programas.

Programa	Consulta
<pre>a :- b1, b2. b1 :- b3. b2. b3. solve(true, 1, 0). solve((A,B), Facts, Rules):- solve(A, F1, R1), solve(B, F2, R2), Facts is F1+F2, Rules is R1+R2. solve(A, Facts, Rules):- clause(A,B), solve(B, Facts, R), Rules is R+1.</pre>	<pre>?- solve(a, X, Y). X = 2, Y = 4. ?- solve(b1, X, Y). X = 1, Y = 2. ?- solve(b2, X, Y). X = 1, Y = 1.</pre>

¿Preguntas?
